241ut240sp BIN THABIT HUSSEIN ABBOD SALMEN

Lab1A-Group8new.pdf

Project

TAI6213

Multimedia University

Document Details

Submission ID

trn:oid:::1:3471187603

Submission Date

Feb 2, 2026, 10:28 PM GMT+8

Download Date

Feb 2, 2026, 11:11 PM GMT+8

File Name

Lab1A-Group8.pdf

File Size

1.8 MB

48 Pages

7,108 Words

39,985 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Table of Contents

1. Introduction 3
1.1 Project Objectives 3
1.2 Problem Statement: Tic-Tac-Toe 4
3. Dataset 5
3.1 Problem Identification 5
3.2 State Representation (LISP Data Structure: Lists) 5
3.3 Data Features and Meaning 5
3.4 Nature of the Target Variable (Finding the Best Choice) 6
4. Data Preprocessing 6
4.1 Variables and Problem Formulation 6
4.2 Functions for State Management and Transitions 7
4.3 Documentation of Representation Choices and Justifications 8
5. Search Method Development 9
5.1 Decision Tree Model Logic 9
5.1.1 State Representation 9
5.1.2 Tree Structure and Branching 9
5.1.3 Terminal States (Leaf Nodes) 10
5.1.4 Search Traversal 10
5.2 Uninformed Search Technique (Implementation of DFS) 11
5.3 Informed Search Technique I (Implementation of Heuristic Search) 15
5.4 Informed Search Technique II (Implementation of Minimax) 17
5.5 Heuristic Logic and Hyperparameter Settings 21
5.5.1 Heuristic Logic Mechanism 21
5.5.2 Hyperparameter Tuning (The Weights) 22
5.5.3 Strategy Justification 22
5.6 Source Code Flow 22
6. Search Method Evaluation 24
6.1 Working Environment (Hardware & Software) 24

6.2 Search Methods Implemented 24
6.3 Test Procedure and Inputs 25
6.4 Evaluation Metrics 26
6.4.1 Accuracy (Optimal Decision Rate) 26
6.4.2 Processing Time 26
6.4.3 Space Used (Estimated Memory) 26
6.5 Evaluation Results 26
6.5.1 Raw Evidence (Console Output) 26
6.5.2 Summary Results Table 30
7. Results and Analysis 30
7.1 Accuracy Analysis 30
7.2 Processing Time Analysis 30
7.3 Internal Node Counter (Mode-Dependent) 31
7.4 Space (Memory) Analysis 32
7.5 Final Interpretation (Which Method Is Best?) 33
Appendix A: Console Output Logs (Allegro CL) 34
Appendix B: 4 Test Runs per Algorithm 45
8. Conclusion 47
8.1 Summary of Project Achievements 47
8.2 Future Improvements and Recommendations 47
9. References 48

1. Introduction

Artificial Intelligence (AI) is used today as the basis for solving complex problems through computer simulation of human decision making via algorithms and data-driven models (Russell & Norvig, 2021). The project described in this paper demonstrates a practical application of AI search techniques in game theory by creating an intelligent agent for the game of Tic-Tac-Toe (Shannon, 1950). The primary purpose of the project is to provide a connection between theoretical AI principles, and their actual implementation, by using the LISP programming language, and focusing on data pre-processing transformation, state-space searching, and formal performance evaluation.

Tic-Tac-Toe is an ideal test case for the strategies being considered, as the game requires AI to search through a finite number of states to find an optimal move while considering the potential response of the opponent. In addition to using a basic graphical user interface, this project includes an optimized analyzer that will provide real-time performance metrics including total execution time, total number of nodes visited, and win probability. The results of the comparison of uninformed explorations, versus informed search, demonstrate how efficiency in the algorithm has a direct impact on the ability of the AI to produce a "perfect play" in a competitive situation.

1.1 Project Objectives

Through this project, we hope to provide an introduction to artificial intelligence through the use of two fundamental components: decision trees and search. The aim of the project will be to use the example of tic-tac-toe to achieve several specific goals.

- **The first goal**, system development, will be to build a working LISP program that can model the game board and provide moves that compete against human players.
- **Secondly,** we will be implementing and optimizing our decision-making algorithms using two different search methods: uninformed search (depth-first search) and informed search (heuristic search + minimax with alpha-beta pruning).
- **Thirdly,** we will acquire first-hand experience transforming a real-life problem into a LISP dataset that can represent a 3x3 grid as a linear list.
- **Fourthly,** we will create an evaluation process to measure how effective our search methods are based on criteria such as the amount of time it took to complete the search, the total number of nodes that have been searched, and how many bytes of memory were utilized.
- **Finally,** we will create a live analysis tool to track the optimality of the AI's decisions based on the ratio of "perfect" decisions to the total number of decisions made (in order to ensure that the agent followed winning or forced-draw paths).

1.2 Problem Statement: Tic-Tac-Toe

Tic-tac-toe is a two-player, zero-sum, adversary game played on a 3x3 grid where each player aims to align three of their marks (X or O) along one row, column, or diagonal. From an AI perspective, to play the game is a search problem in that every move made by either player results in a transition to a new state within some tree like structure (usually referred to as the decision tree).

There are a number of key issues the system has to resolve:

- **State Representation** - The current state of the game must be represented as the initial state of nine empty cells within the state space. In LISP, for example, it would be represented by the expression (**NIL NIL NIL NIL NIL NIL NIL NIL NIL**).
- **Optimal Decision Making** – The AI must determine the best available move with respect to maximizing its own survival (to prevent itself from losing) and to help it win (i.e. achieving maximum score).
- **Adversarial Complexity** – The agent must account for its opponent's move as well, and therefore the agent has to use algorithms such as Minimax to minimize its potential loss while maximizing its own score.
- **Resource Management** – The search must be performed efficiently. The system will keep track of "Nodes Visited" and "Memory Est" to make sure that the search does not get delayed on long and/or redundant decision paths.

In order to control the states of the game, this project defines the following sets of possible configurations as goal criteria:

```
(defparameter *initial-board* '(NIL NIL NIL NIL NIL NIL NIL NIL NIL))
(defparameter *winning-lines* '((0 1 2) (3 4 5) (6 7 8) (0 3 6) (1 4 7) (2 5 8) (0 4 8) (2 4 6)))
```

Figure 1 : Initial, and winning states

3. Dataset

The dataset associated with this project consists of the structural model of the Tic-Tac-Toe game environment. The dataset has been created specifically with respect to processing in the LISP programming language and involves changing the physical representation of the game, being a 3-by-3 grid, to a LISP-readable form (e.g., a LISP-type list) in order for the AI to carry out a state-space search and evaluate the state of the board.

3.1 Problem Identification

The principal problem associated with this project is the creation of an intelligent agent for solving the Tic-Tac-Toe puzzle, which is a classic problem utilized in Artificial Intelligence to demonstrate the effectiveness of the different types of search and decision-making algorithms. The development of the intelligent agent to solve the Tic-Tac-Toe puzzle involved identifying the problem along with generating a suitable dataset regarding the use of search methods and decision logic. As a zero-sum game, Tic-Tac-Toe presents a challenge for the AI and involves evolving from an initial state of an empty board through to a terminating state that maximizes the agent's expected utility in terms of a win or forced draw.

3.2 State Representation (LISP Data Structure: Lists)

The goal of this design is to create a representation of the game state in a linear list (the simplest way to represent game states in LISP) for working with searching.

- **Game Board:** the board is represented using a parameter named *initial board*, which holds a list of nine (NIL) values to represent empty squares on a tic-tac-toe board.
- **Winning Condition:** the *winning lines* parameter contains a subset of the winning lines of the board (the actual indices) as a list of lists.
- **Transitions Between States:** the method make-move is the way to implement the transitions between states; it does this by using copy-list to create a copy of the current board (original).

```
(defparameter *initial-board* '(NIL NIL NIL NIL NIL NIL NIL NIL NIL))
(defparameter *winning-lines* '((0 1 2) (3 4 5) (6 7 8) (0 3 6) (1 4 7) (2 5 8) (0 4 8) (2 4 6)))
```

Figure 2 : Initial, Winning States

3.3 Data Features and Meaning

For the purposes of this document, the features of the data set will be represented by the individual cells of a (3x3) grid, and the values of the corresponding features will indicate the current owner of the cell.

- **Indices (0-8):** The indices in the original list represent a unique position on the board, which corresponds to the 1-9 number representation that will be displayed to the player.
- **NIL:** represents an empty square, meaning that a mark can still be added to that square.
- **X and O:** the two possible markings on the squares of the grid, representing X = human player, O = AI agent.

3.4 Nature of the Target Variable (Finding the Best Choice)

In a search-based problem, there is a target to achieve (an optimal move or "best choice") by the computer. Unlike problems that have one "static label," this problem has a dynamic target because the computer needs to find the index (from 0 to 8) that will lead to the best possible terminal state for itself. When determining which move will give it the maximum probability of winning or guarantee that it will at least force a draw, the AI uses either an informed or uninformed searching method to evaluate the target variable.

In addition, the analyze-move function uses the target variable as a means of identifying whether or not they can make the best move, based on the corresponding score of the score will have either:

- **Winning Path**: a 100% chance of winning.
- **Forced Draw**: 50% chance of remaining neutral outcome.
- **Losing Path**: has a 0% chance of being successful.

4. Data Preprocessing

To make use of LISP to solve the Tic-Tac-Toe problem, the game environment has to be defined and organized in a way that will allow the search algorithms to navigate through and manipulate the information it contains efficiently by defining the game board, the players, and the rules associated with the game as a set of symbolic representations.

4.1 Variables and Problem Formulation

A state-space search problem, the Tic-Tac-Toe game, allows the AI agent to start with an empty board (the initial state) and make moves through state transitions into the terminal game state (a state with no remaining moves and cannot be further expanded). The state representation is a 9-element linear list of the cells of the 3x3 board, which means that the state management is simple and utilizes the symbolic processing strengths of LISP.

Here are the main game variables that define the game:

- **Initial state of the game board:** Empty board (3x3) of NIL's, each of the nine cells (cells 0-8) is available for the next move.

```
(defparameter *initial-board* '(NIL NIL NIL NIL NIL NIL NIL NIL NIL))
```

Figure 3 : Intiial state

- **Winning conditions:** A predefined data set containing the eight possible index combinations (rows, columns, diagonals) that constitute a win.

```
(defparameter *winning-lines* '((0 1 2) (3 4 5) (6 7 8) (0 3 6) (1 4 7) (2 5 8) (0 4 8) (2 4 6)))
```

Figure 4 : Winning state

- **Performance metrics:** Variables to track the search algorithm's operational efficiency and accuracy.

```
;; --- GLOBAL STATS TRACKERS ---
(defvar *nodes-visited* 0)
(defvar *total-ai-moves* 0)
(defvar *optimal-ai-moves* 0)
```

Figure 5 : Performance metrics

4.2 Functions for State Management and Transitions

Functions are implemented to perform moves, display the Tic-Tac-Toe game environment, and check for terminal states.

- **Display board:** Presents the linear list of the board contents (nine index entries of either "X", "O", or "NIL" into a standard grid format (3 rows x 3 columns) for the user's convenience.

```
;; --- UTILITY FUNCTIONS ---
(defun display-board (b)
  (format t "~% ~a | ~a | ~a" (or (nth 0 b) "1") (or (nth 1 b) "2") (or (nth 2 b) "3"))
  (format t "~%-----------")
  (format t "~% ~a | ~a | ~a" (or (nth 3 b) "4") (or (nth 4 b) "5") (or (nth 5 b) "6"))
  (format t "~%-----------")
  (format t "~% ~a | ~a | ~a ~%" (or (nth 6 b) "7") (or (nth 7 b) "8") (or (nth 8 b) "9")))
```

Figure 6 : Display Board

- **Transition model (make-move):** Produces a new board state by copying the list of index values (the values from the original board state) and applying the player's "mark" (either "X" or "O") in the specified cell index on the copied linear list. The copied list will not change the original board state (allowing for backtracking to the original state during a search).

```
(defun make-move (board index player)
  (let ((new-board (copy-list board)))
    (setf (nth index new-board) player)
    new-board))
```

Figure 7 : Make move

- **Goal test (check-win):** Searches for any existing winning combination for the current board state.

```
(defun check-win (board player)
  (some (lambda (line) (every (lambda (idx) (eq (nth idx board) player)) line)) *winning-lines*))
```

Figure 8 : Check win

- **Successor Generation:** Enumerates the successor states (NIL positions) that can be reached from the current state while constructing the game tree.

```
(defun get-successors (board player)
  (let ((successors '()))
    (dolist (i '(4 0 2 6 8 1 3 5 7))
      (when (null (nth i board))
        (push (make-move board i player) successors)))
    (nreverse successors)))
```

Figure 9 : Get successors

4.3 Documentation of Representation Choices and Justifications

Data structure and function choices were made with the goals of maximizing search efficiency and maintaining integrity of the data.

- **Choice of Lists:** The project uses lists instead of arrays for fixed-size data structures, like many systems do. The choice takes advantage of LISP's ability to copy lists and recurse through lists (Graham, 1995). Both features are necessary when generating and searching through thousands of nodes in a game tree.

- **NIL Placeholders:** NIL is used to represent empty squares in the board (that is, the squares that may accept pieces) to enable the system to quickly and easily test for valid moves by checking whether a square is empty using the null predicate. This means that only valid moves (legal branches) will be examined by the search algorithm.
- **Uniform Path Cost:** Each move has a uniform path cost of one, making cumulative cost calculations much easier to calculate in more complex searches, such as Minimax.
- **Symbolic Markers:** Instead of using strings or integers, 'X' and 'O' will be used to identify pieces on the board. This is consistent with LISP's symbolic nature, making it easier for the logic engine to read the data, while at the same time causing less processor overhead (Seibel, 2005).

5. Search Method Development

5.1 Decision Tree Model Logic

In this work, Tic, Tac, Toe has been described as a state, space search problem that can be represented as a decision tree. Such a structure provides the basis for an AI system to explore the potential next game states methodically and make the best move.

5.1.1 State Representation

The most basic element of our decision tree is the "state", which is a snapshot of the game board now of a player's turn.

- **Data Structure:** In the LISP code of our project, a state is a list of 9 elements (e.g., (NIL NIL 'X NIL 'O ...)).
- **Mapping:** The numbers 0 to 8 indicate places in a 3*3 grid which has been numbered row, wise from the top left to the bottom right corner.
- **Values:** Each item on the list can be one of three things:
    - NIL: This is an empty square on the board (one can play there).
    - 'X: It is a spot taken by player one.
    - 'O: It is a spot taken by player two (or the AI).

5.1.2 Tree Structure and Branching

A decision tree is built on; the fly starts at Root Node which is the board without AI move.

- **Branching Factor:** At any state, the number of children the node has will be equal to the number of NIL elements in the list. E.g., if the board has 5 empty spots, then the current node will have 5 different child nodes.
- **Edges:** The edges in the tree refer to the moves (actions) that the player has done in the game.

5.1.3 Terminal States (Leaf Nodes)

The expansion of the tree will stop once it reaches a Terminal State. Based on our

check-win and board-full-p functions, the conditions that define a terminal leaf are:

1. **Win:** One of the players has a full row, column, or diagonal line (as per the global parameter (winning-lines).
2. **Draw:** There is no NIL values left on the board and no winner.

5.1.4 Search Traversal

The AI will traverse the tree to find a solution to the problem.

- **Uninformed Search (DFS):** The method goes down a single path deep to a terminal node. Only when it is forced to, the method rolls back a few steps.

- **Informed Search (Minimax/Heuristic)** generates the tree up to a certain depth and at the same time it assigns a numerical "score" to the leaf nodes (e.g., +10 for a win, -10 for a loss). These scores are then propagated back up the tree to the root, thus allowing the AI to pick the branch with the highest probability of success.

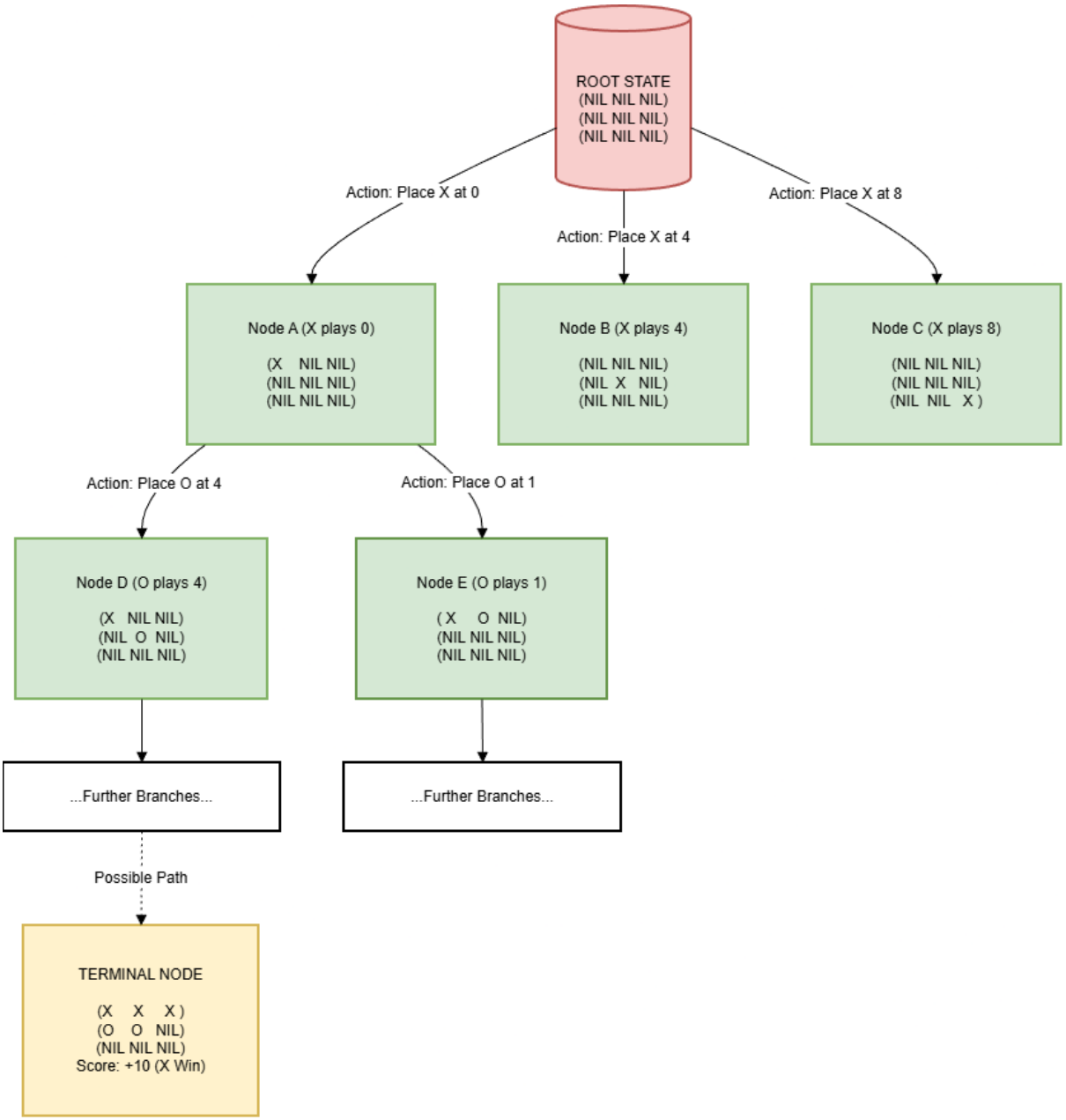


Figure 10 : Partial Decision Tree Representation for Tic-Tac-Toe Game States

5.2 Uninformed Search Technique (Implementation of DFS)

In the uninformed search part of this project, we designed a **Depth, First Search (DFS)** algorithm. The process involves searching the decision tree by going as deep as possible along each branch before backtracking. It is considered "uninformed" (or "blind") because it does not use any heuristic function or domain knowledge to estimate the "goodness" of a non, terminal state; rather, it depends on terminal checks (win/loss/draw) only to decision-making.

5.2.1 Algorithm Mechanism (Recursive Check)

The heart of our DFS is the dfs-check function, which acts as the recursive engine. It helps in visualizing the game state starting from the current one and deciding whether a particular path ends in the target victory or not.

- **Terminal Tests:** Every time the recursion goes deeper, the first thing the algorithm does is check if the current state represents a leaf node using these three conditions:

  1. **Victory:** If (check-win board target-player) returns true, then the path is considered a successful one.

  2. **Defeat:** If (check-win board opponent) returns true, then the path is discarded (pruned) right away so that the AI will not choose losing moves.

  3. **Draw:** If (board-full-p board) returns true without a winner, then the path is considered a neutral one (returning nil in our logic to give precedence to winning paths over draws).

- **Recursive Expansion:** In the case of no terminal state being reached, the function recursively calls get-successors to produce the next layer of board states. Subsequently, it applies the LISP some function to recursively call the dfs-check on these children. This in practice goes down the tree depth, first, switching the current-player at each level to simulate alternating turns.

Code Implementation:

```
(defun dfs-check (board current-player target-player)

  (incf *nodes-visited*)  ;; Track complexity

  (cond

    ((check-win board target-player) t) ;; Target won

    ((check-win board (if (eq target-player 'X) 'O 'X)) nil) ;; Opponent
won

    ((board-full-p board) nil)  ;; Draw (treated as non-win)

    (t (let ((next-p (if (eq current-player 'X) 'O 'X)))

         ;; Recursively check successors until a True path is found
```

5.2.2 Move Selection Strategy

The dfs, move function is the engine that drives the AI's turn. Unlike the recursive checker, this function oversees the moment, to, moment decision of which move should be played on the actual game board.

- **Search & Selection:** It goes through the immediate moves available (generated by get-successors) and sends each one to dfs-check. The first move that produces a "winning" confirmation is set aside as best, move.
- **Robustness (The Fallback):** A crucial aspect of our implementation is the safety fallback. As DFS looks for a guaranteed win, it can return nil if all the available ways lead to a draw or a loss (which is typical in Tic, Tac, Toe against optimal play). To save the program from getting stuck or crashing, we put a fallback condition:

(if best-move best-move (first (get-successors board player)))

This guarantees that even in a "hopeless" situation, the AI will still perform a valid move.

Code Implementation:

```
(defun dfs-move (board player)
  (let ((best-move nil))
    ;; Try to find a move that returns true from dfs-check
    (some (lambda (successor)
            (if (dfs-check successor (if (eq player 'X) 'O 'X) player)
                (setf best-move successor)))
          (get-successors board player))
    ;; Fallback: If no winning move, pick the first available one
```

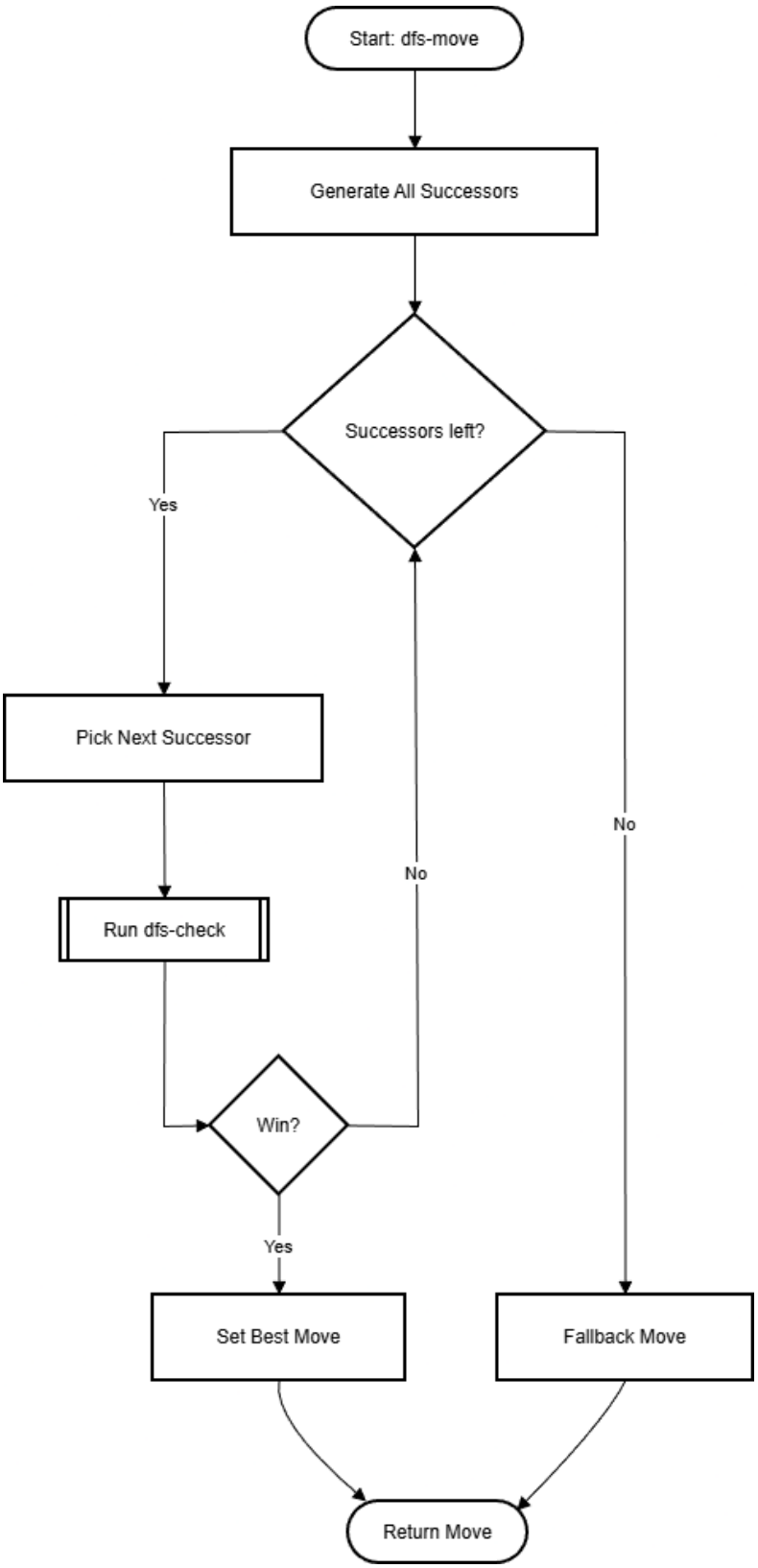


Figure 11: Flowchart of the Uninformed Depth-First Search (DFS) Algorithm

5.3 Informed Search Technique I (Implementation of Heuristic Search)

For the first informed search part of the project, we chose to implement a **Heuristic Search** (more specifically **Greedy Best-First Search**). The algorithm "goodness" of a few next states first by looking at their immediate next moves to the decision tree and described that process as looking at the immediate next moves of the next states to the decision tree. It is called " informed " because it uses a domain, specific evaluation function to estimate the strategic value of a move, rather than relying blindly on terminal checks.

5.3.1 How Algorithm Works (Heuristic Evaluation)

The heart of our heuristics approach is the evaluation, heuristic function. This function is a scoring engine.
What it does is mean that it takes in a certain board state and returns a numeric value that reflects the strategic situation of that board.

- **Scoring Logic:** The function computes a score that is a sum of the weighted features:
  - **Immediate Win:** (+10, 000) This is given when the move leads to a win for the AI.
  - **Immediate Loss**: (-10, 000) This is given when the move lets the opponent win on his/her next turn.
  - **Threats**: (+100 for AI, ,1000 for Opponent) Two identical symbols in a row with one open spot constitute a "threat". We attribute a much bigger negative value to opponent threats (-1000) to make the AI block defensively.
  - **Positional Bonus:** (+10) It is a reward for taking the center square, which is an important strategic feature of Tic, Tac, Toe.
- **Greedy Selection:** This function is not a deep recursive one. It assesses only the board's current static state; thus, it is computationally light as compared to Minimax.

Code Implementation:

```
(defun evaluate-heuristic (board player)
  (let ((opp (if (eq player 'X) 'O 'X)))
    (cond ((check-win board player) 10000)   ;; Immediate Win
          ((check-win board opp) -10000)     ;; Immediate Loss
      (t (+ (* (count-threats board player) 100)
            (* (count-threats board opp) -1000) ;; Heavy penalty for blocking
            (if (eq (nth 4 board) player) 10 0)))))) ;; Center control bonus
```

5.3.2 Move Selection Strategy

The heuristic, search function is the main function for the AI's move. It uses the "Greedy" method by picking the move that is locally optimal at each step.

- **Search & Sort:** The function creates all legal successors with the help of get-successors. Next, it maps the score of each successor obtained from heuristic evaluation using evaluate-heuristic.
- **Best-First Decision:** The moves are sorted by their scores in descending order. Hence, the function returns the highest-scoring move from the sorted list and returns it immediately (cdar of the sorted list). This way, AI is always guaranteed to make the best move now.

Code Implementation:

```
(defun heuristic-search (board player)

  (incf *nodes-visited*)

  (let* ((succ (get-successors board player))

         ;; Map each successor to its score: ((Score1 . State1) (Score2 .
State2)...)

         (scored (mapcar (lambda (s) (cons (evaluate-heuristic s player) s))
succ)))

    ;; Sort by score (Highest first) and return the state associated with the
best score

    (if scored (cdar (sort scored #'> :key #'car)) nil)))
```

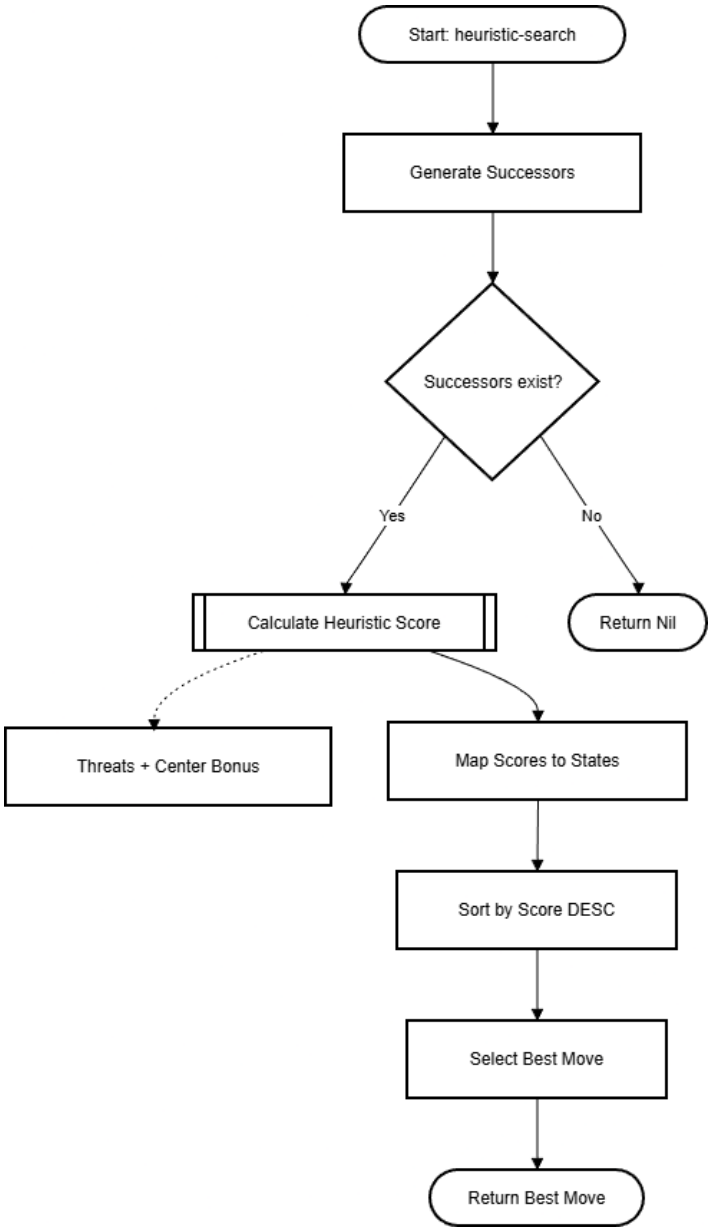


Figure 12: The linear process of the Greedy Heuristic search!

5.4 Informed Search Technique II (Implementation of Minimax)

We employed the Minimax Algorithm for the second informed search module of this project. With the help of this method the decision tree is scrutinized extensively to identify the theoretically best move. It is an "informed" method as the recursively computed scores are based on the assumption of the opponent playing a perfect game thus minimizing the success of the AI (Russell & Norvig, 2021).

5.4.1 Description of Algorithm (Min, Max Recursion)

At the heart of our Minimax code lies the minimax method. Its main function is to transfer values from the final states in the game tree to the current state finding the most desirable result.

- **Recursive Simulation:** Through alternating turns the method is continually self, invoked as follows:
  - **Maximizing (AI's Turn):** Attempts to identify the move with the maximum score (max).
  - **Minimizing (Opponent's Turn):** Attempts to identify the move with the minimum score (min), thus simulating a perfect opponent.
- **Terminal Valuation:** Once the terminal state is attained, a definite score is given
  - **Win:** 10 - depth (Promotes winning in the shortest possible time).
  - **Loss:** depth - 10 (Promotes postponing loss for as long as possible).
  - Draw: 0.

Code Implementation:

```
(defun minimax (board depth is-max)
  (if (or (check-win board 'X) (check-win board 'O) (board-full-p board))
      (get-minimax-score board depth) ;; Base Case: Return calculated score
      (if is-max
          ;; Maximizing Player (AI): Find highest score
          (let ((best -1000))
            (dotimes (i 9) (when (null (nth i board))
              (setf best (max best (minimax (make-move board i 'O) (1+ depth)
nil)))))
            best)
          ;; Minimizing Player (Opponent): Find lowest score
          (let ((best 1000))
            (dotimes (i 9) (when (null (nth i board))
```

5.4.2 Move Selection Strategy

The minimax-search function is like a driver. It starts the recursive process for each move available to find the best one.

- **Iteration & Comparison:** It goes through all the immediate neighboring moves. For each move, it invokes the minimax function (at depth 0) to retrieve the "true value" of that move.

- **Optimal Selection:** I keep a record of the highest score that has been encountered so far. If a move's score calculated is greater than the current best, the best-move is then updated. This way the AI is choosing the move which mathematically guarantees the best outcome.

Code Implementation:

```
(defun minimax-search (board player)
  (incf *nodes-visited*)
  (let ((best-move nil) (best-score -1000))
    (dotimes (i 9)
      (when (null (nth i board))
        ;; Call Minimax on this potential move
        (let ((score (minimax (make-move board i 'O) 0 nil)))
          (when (> score best-score) (setf best-score score) (setf best-move
(make-move board i 'O))))))
    best-move))
```

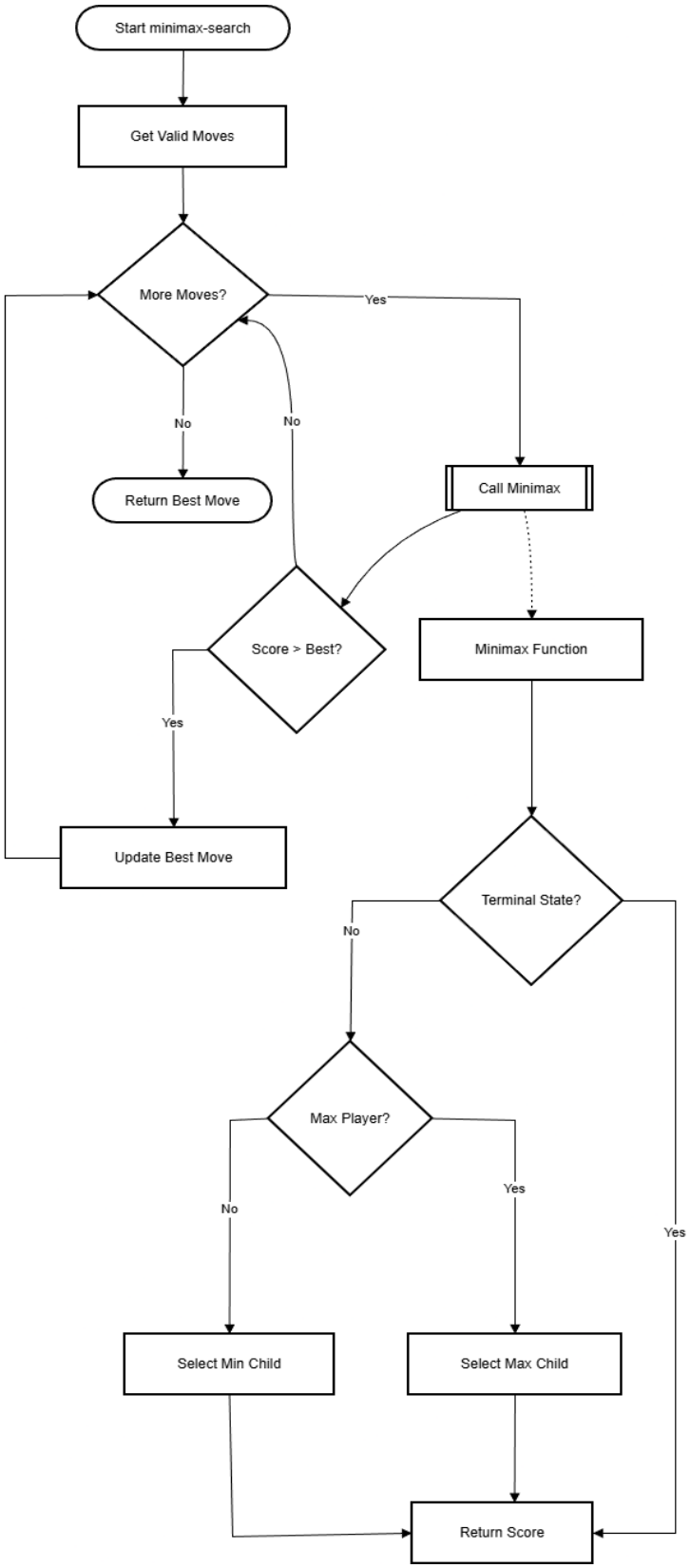


Figure 13: Minimax Algorithm Flowchart

5.5 Heuristic Logic and Hyperparameter Settings

One of the important ways to make **Greedy Best-First Search** behave smartly but at the same time not deeply recursive is through an accurately made heuristic evaluation function. Such a function evaluates the strategic value of a particular board situation by internally assigning scores to different facets of the game with various weights. Changing these weights (hyperparameters) is the way we gave the AI the character of primarily wanting to survive and after that look for opportunities to win.

5.5.1 Heuristic Logic Mechanism

The core of the matter is in the evaluate-heuristic function. Without an exhaustive search for the game tree (which is very computationally expensive), this function picks a fixed "score" for a particular board state by looking at three main things:

- **Terminal Status:** Firstly, it checks if the state is already a Win or Loss. These are absolute values that completely ignore other logic.
- **Threat Detection:** It searches for "threats" on board. A threat is one of the lines (row, column, or diagonal) that have two identical symbols and one empty spot. It implies a "next move" winning condition.
- **Positional Strategy:** It verifies whether the AI has claimed the Center Square (Index 4). In Tic, Tac, Toe, the center is the most advantageous spot as it participates in the highest number of winning lines (4 lines).

Code Implementation:

```
(defun evaluate-heuristic (board player)

  (let ((opp (if (eq player 'X) 'O 'X)))

    (cond ((check-win board player) 10000)   ;; Condition 1: Immediate
Win

          ((check-win board opp) -10000)     ;; Condition 2: Immediate
Loss

      (t (+ (* (count-threats board player) 100)

            (* (count-threats board opp) -1000) ;; Priority: Block
Opponent

            (if (eq (nth 4 board) player) 10 0)))))) ;; Positional Bonus
```

5.5.2 Hyperparameter Tuning (The Weights)

The performance of the Heuristic Search totally depends on the numerical values of the features listed above. These values are the hyperparameters of our model. We adjusted these weights in such a way that our agent behaves "Defensively", it is more concerned with blocking the opponent than making its own attacks.

Table 1: Hyperparameter Tuning table

5.5.3 Strategy Justification

The single most important architectural decision in this reasoning is the difference between the AI Threat (+100) and the Opponent Threat (-1000).

- **Why -1000?** If we had given them equal weights (for example, +100 and -100), the AI might have decided to generate its own threat rather than block the opponent's threat and hence it would be defeated in the very next move.
- **The Decision:** Putting the opponent's threat penalty ten times bigger, we compel the Greedy algorithm to "panic" and block the opponent's two in a row, thus it plays a solid defensive game even without using the recursive look, ahead.

5.6 Source Code Flow

The flowchart below explains the general steps of the source code. It shows how the system starts, the human player (X) and the AI (O) turns are processed, and how the game state transitions are managed until a final state (Win, Loss, or Draw) is reached.

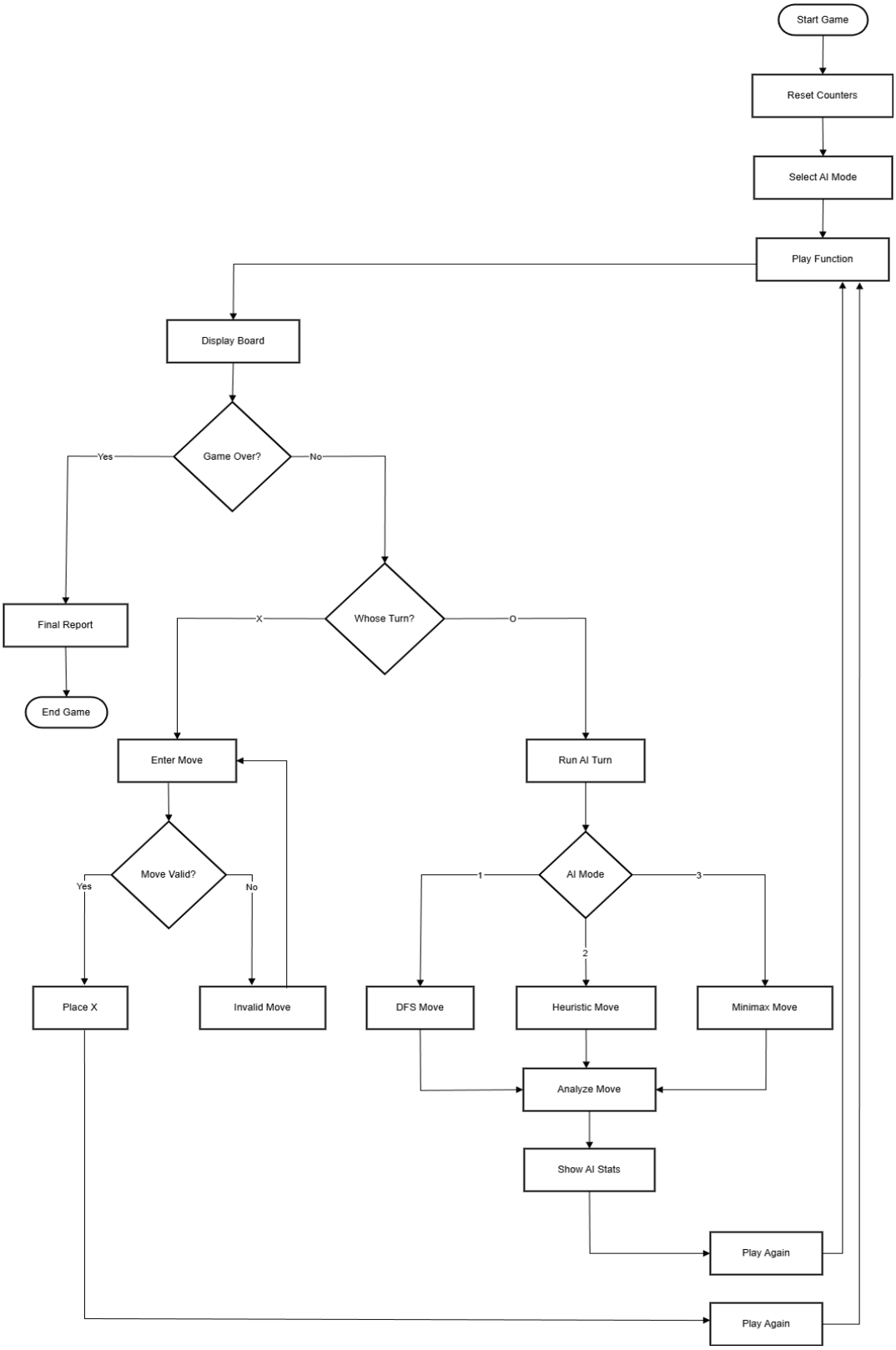


Figure 14: Source Code Flow Chart

6. Search Method Evaluation

6.1 Working Environment (Hardware & Software)

Table 2: Working Environment Used for Evaluation

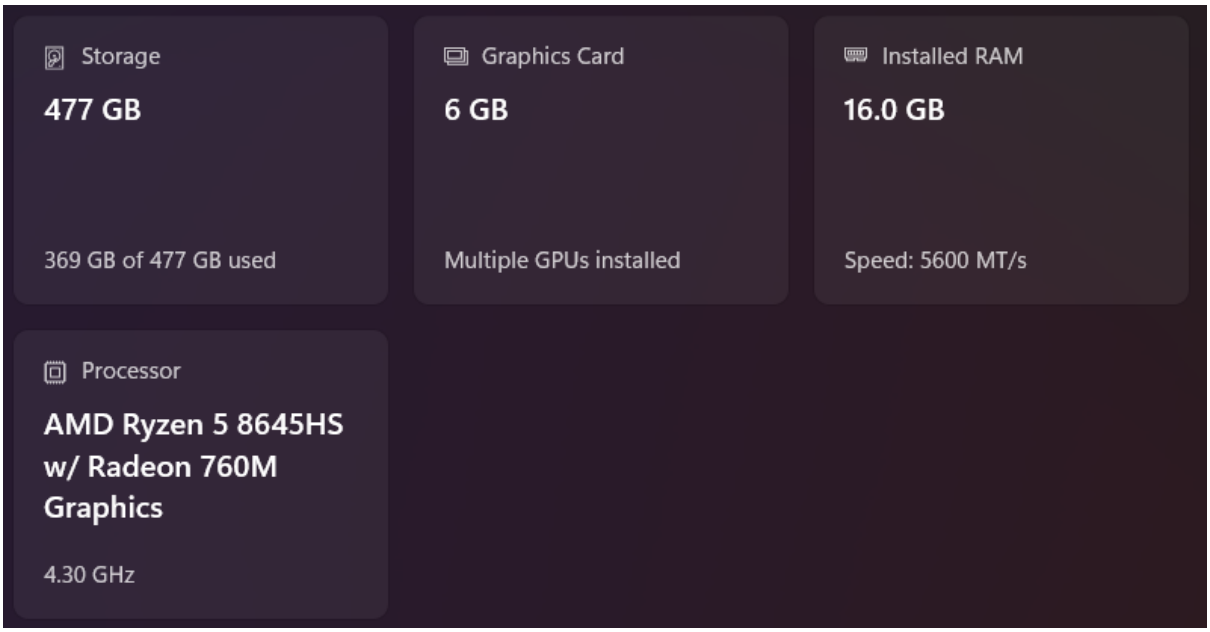


Figure 15: Hardware specifications used for evaluation (CPU, RAM speed, and GPU memory).

6.2 Search Methods Implemented

Tic Tac Toe in this project has been modelled as a search problem; the nodes represent the current state of the board, and the edges represent the legal moves. The current state of the board is stored as a list of nine cells, with each cell being either NIL for empty cells, X or O for the marks made by the two players. The successor states for each move are generated by a predefined vertex order. The order of priority will first create all successor states due to the center cell (middle), then all the corner cells will be created next, and finally all the edge cells will be created last.

The following search methods were evaluated:

- **Uninformed Search — Depth-First Search (DFS)**
  DFS does not model optimal counter-play, so it may select moves that the Minimax referee evaluates as suboptimal or losing. Although DFS can appear fast due to fewer explored nodes, its decision quality is inconsistent compared to the heuristic and Minimax methods.

- **Informed Search — Heuristic Greedy Search**
  To evaluate all possible game board states that could occur from a given move, heuristic search uses a scoring method to decide the best move by taking into account three main areas of scoring (e.g., making threats against the opponent, blocking opponent threats and controlling the center of the board), and then selecting a move based on the highest score without further recursive evaluations for each move. The scoring and the choice of a move to perform, as found in heuristic search, are accomplished using heuristic searching and evaluating functions. In this implementation, the node counter is an internal counter; in heuristic mode, it increments once for each successor state evaluated during greedy move selection (the number of legal moves from that board state), so the console typically reports Nodes > 1 in early and mid-game.

- **Informed Reference — Minimax with Alpha-Beta Pruning (Minimax+AB)**
  Minimax+Alpha-Beta applies a recursive method of evaluation to read the game tree to terminal states: win/lose/draw and utilizes alpha-beta pruning to eliminate branches that cannot improve upon the final decision. Minimax+AB is further employed during the referential method used for accuracy calculations to assess the Perfect Decision (i.e., the best possible Minimax score) for each AI's move.

Lastly, in addition to the above methods, a performance analysis application is provided to provide information on the AI operations, including time, number of nodes visited, estimated memory, and correctness/accuracy of AI decisions.

6.3 Test Procedure and Inputs

The evaluation was performed by executing the application under each mode and collecting all the statistics for each AI turn printed.

The performance metrics collected for each AI turn included:

- Processing Time per AI moves (in seconds)
- Nodes Visited per AI move
- Estimated Memory Usage (printed out by the analyzer)
- Win Chance Label (Winning / Draw / Losing Path)
- Final Accuracy Printed in Final Performance Report

These performances were evaluated across naturally generated in-game states during play, covering early, mid, and late game states.

Note*: the "Nodes" counter the analyzer provides is an internal counter and is implemented differently for each of the application modes. DFS and Minimax + AB will both increment the counter as the algorithm recursively explores the game tree. The Heuristic mode will increase the counter once for each successor state evaluated during move selection (one increment per candidate move scored). The output of the Node values is valid but should be interpreted differently when compared because of the difference in the counting method used in each of the*

modes.

6.4 Evaluation Metrics

The following metrics are used to evaluate the search methods:

6.4.1 Accuracy (Optimal Decision Rate)

In this project, each AI turn produces exactly one move decision. A Perfect Decision is defined as an AI move whose Minimax referee score matches the best achievable Minimax score among all legal AI moves from the same board state. The system tracks the total number of AI moves and the number of Perfect Decisions, then calculates:

$$Accuracy\ (\%) = (Perfect\ Decisions\ /\ Total\ AI\ Moves) \times 100$$

This value is printed in the “FINAL PERFORMANCE REPORT” at the end of each game.

6.4.2 Processing Time

The Time for each AI Move is recorded using get-internal-run-time and represents time in seconds and is printed by the analyzer’s display. Any very rapid AI Moves will show up as "0.0" because they are less than the resolution of the Timer used in the Analyzer.

6.4.3 Space Used (Estimated Memory)

Utilization of Space is indicated by the program's Memory Est printout (printed Memory Est). This value represents the number of nodes captured and their overall estimated costs per node as created within the analyzer program. The Memory Est value is displayed in KB (with appropriate formatting/rounding) to facilitate console readability. In this implementation, Memory Est (KB) is calculated as: Nodes × 128 bytes ÷ 1024 (≈ Nodes × 0.125 KB).

6.5 Evaluation Results

6.5.1 Raw Evidence (Console Output)

All of the data for these performance metrics was derived directly from output logs of the Allegro CL console.

```
[AI STATS]
 > Algo:          DFS
 > Time:            0.0 s
 > Nodes:         9
 > Memory Est:    1. KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

```
[AI STATS]
 > Algo:          DFS
 > Time:            0.0 s
 > Nodes:         4
 > Memory Est:    .5 KB
 > Win Chance:    0% (Losing Path)
-----------------------------
```

DFS (AI STATS)

```
=========================================
      FINAL PERFORMANCE REPORT
=========================================
 Total AI Moves:      2
 Perfect Decisions:   1
 AI FINAL ACCURACY:   50.%
=========================================

>>> RESULT: X WINS! <<<
```

DFS (REPORT)

```
[AI STATS]
 > Algo:          HEURISTIC
 > Time:            0.0 s
 > Nodes:         8
 > Memory Est:    1. KB
 > Win Chance:    50% (Forced Draw)
-----------------------------

[AI STATS]
 > Algo:          HEURISTIC
 > Time:          0.016 s
 > Nodes:         6
 > Memory Est:    .7 KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

```
[AI STATS]
 > Algo:          HEURISTIC
 > Time:          0.015 s
 > Nodes:         4
 > Memory Est:    .5 KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

```
[AI STATS]
 > Algo:          HEURISTIC
 > Time:          0.016 s
 > Nodes:         2
 > Memory Est:    .2 KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

Heuristic (AI STATS)

```
=========================================
      FINAL PERFORMANCE REPORT
=========================================
 Total AI Moves:      4
 Perfect Decisions:   4
 AI FINAL ACCURACY:   100.%
=========================================

>>> RESULT: DRAW! <<<
```

Heuristic (REPORT)

```
[AI STATS]
 > Algo:          MINIMAX+AB
 > Time:          3.172 s
 > Nodes:         5119
 > Memory Est:    600. KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

```
[AI STATS]
 > Algo:          MINIMAX+AB
 > Time:          0.281 s
 > Nodes:         433
 > Memory Est:    50. KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

```
[AI STATS]
 > Algo:          MINIMAX+AB
 > Time:          0.031 s
 > Nodes:         34
 > Memory Est:    4. KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

```
[AI STATS]
 > Algo:          MINIMAX+AB
 > Time:            0.0 s
 > Nodes:         4
 > Memory Est:    .5 KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

Minimax+AB (AI Stats)

```
=========================================
      FINAL PERFORMANCE REPORT
=========================================
 Total AI Moves:      4
 Perfect Decisions:   4
 AI FINAL ACCURACY:   100.%
=========================================

>>> RESULT: DRAW! <<<
```

Minimax+AB (REPORT)

(Full logs are included in Appendix A.)

6.5.2 Summary Results Table

Table 3: Performance Summary (Single-Game Run per Algorithm, Allegro CL)

***Note**: The numbers found in Table 2 have been calculated from a single run through each algorithm provided in Appendix A.*

7. Results and Analysis

***Note**: The discussion in Section 5 is based on the representative single-run results reported in Table 2. Additional repeated test runs (four per algorithm) are provided in Appendix B to validate the overall trend. Minor differences in DFS vs Heuristic timing may occur because many moves are completed below the timer resolution and are printed as 0.0 s.*

7.1 Accuracy Analysis

According to Table 2, DFS achieved 50% accuracy, with one Perfect Decision out of two AI moves (the second move was assigned by the Minimax referee the label "0% (Losing Path)", indicating it did not match the best achievable minimax score for that game state). In the single recorded run of the heuristic method, all AI moves matched the best Minimax referee score for the evaluated states, resulting in 100% accuracy for those states; however, this result is specific to the tested board configurations and does not guarantee the same performance across all board layouts.

7.2 Processing Time Analysis

In the representative run (Table 2), DFS and Heuristic both had several moves printed as 0.0 s, which indicates that those decisions completed below the timer resolution in this environment. Across that same run, the Heuristic method recorded non-zero decision times of 0.015–0.016 s (avg 0.012 s, max 0.016 s). DFS averaged 0.000 s (max 0.000 s) because both observed DFS turns fell below the measurable range. Minimax + AB was slower, especially in the early game, because it recursively evaluated a much larger portion of the game tree even with alpha-beta pruning (avg 0.871 s, max 3.172 s).

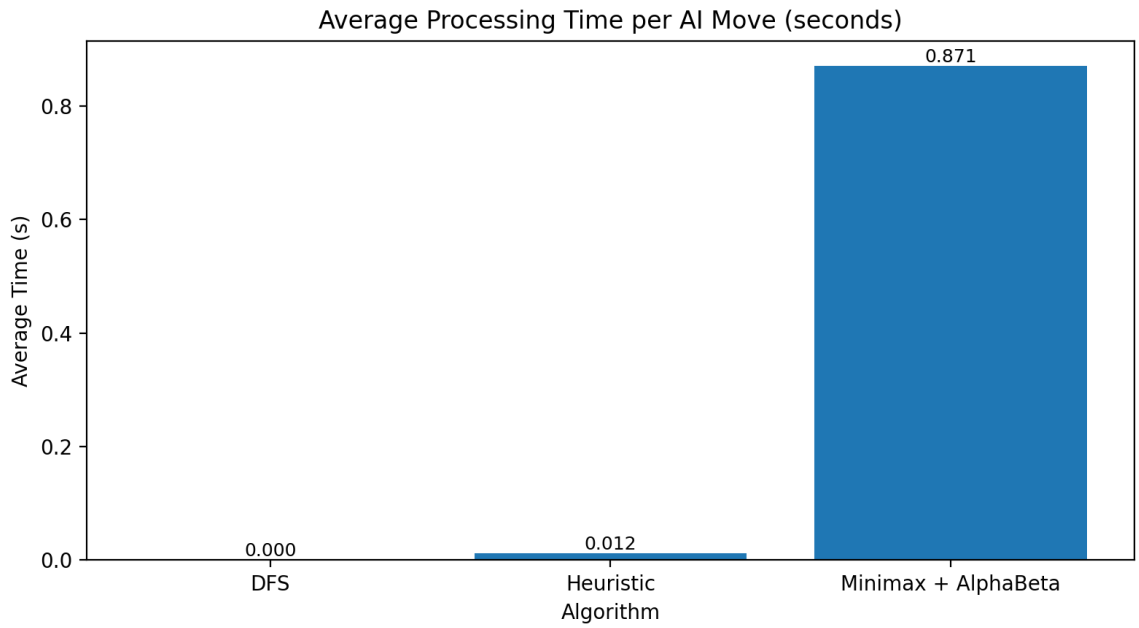


Figure 16: Average AI decision time across DFS, Heuristic, and Minimax+AB.

7.3 Internal Node Counter (Mode-Dependent)

Per the nodes visited counter in the program, there were:

- DFS - 6.5 nodes (Max 9)
- Heuristic - 5.0 nodes (Max 8)
- Minimax+AB - 1397.5 nodes (Max 5119)

The reason Minimax + AB had the most nodes visited was due to a large search tree generated by early game positions, and while alpha-beta pruning would eliminate many branches from

consideration, the remaining search was still significantly greater than either DFS or the greedy heuristic.

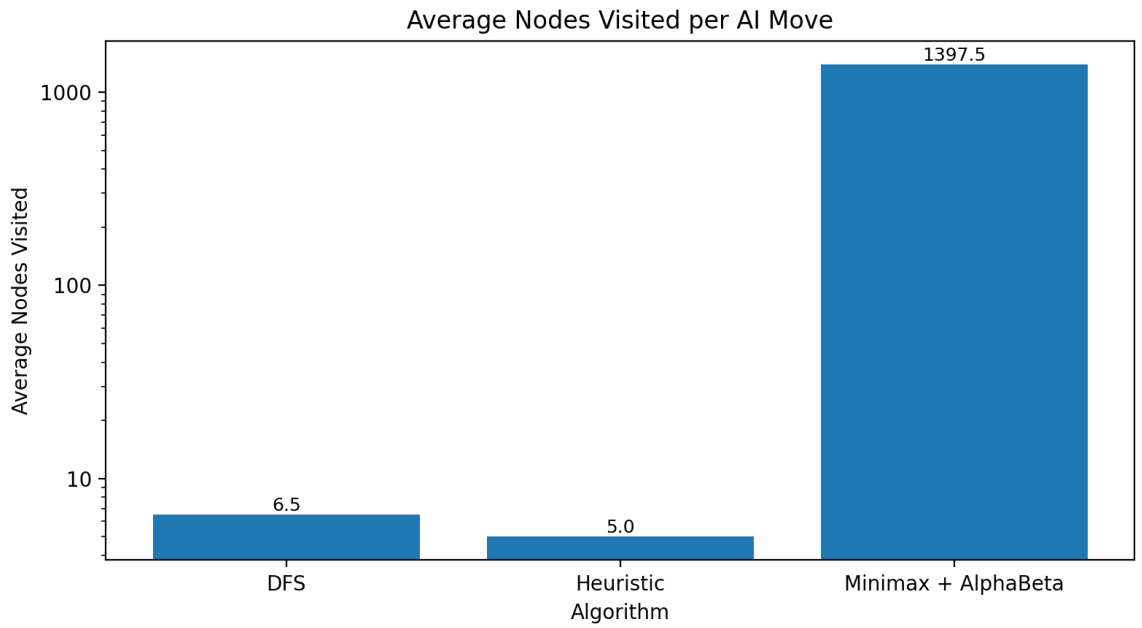


Figure 17: Average nodes visited per move; higher values indicate heavier search effort.

7.4 Space (Memory) Analysis

The memory estimates of the various algorithms have a strong correlation with those of their respective printed node counts. The analyzer's Memory Est is calculated directly from the number of nodes captured and using a preset assumption of cost per node, and the memory value is printed in kilobytes for clarity. In the representative run (Table 2), DFS averaged 0.75 KB while Heuristic averaged 0.60 KB; both remain small because they evaluate few states per move. Minimax + AlphaBeta produced the highest memory footprint (163.63 KB average; 600.0 KB max) due to conducting much deeper recursive searches in early-game positions.

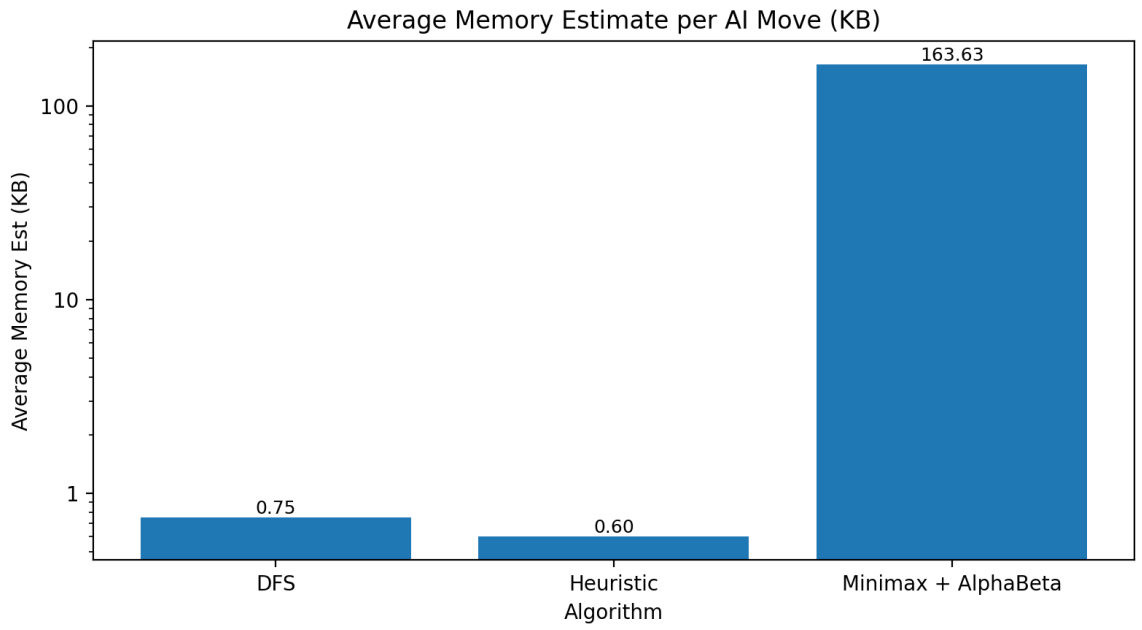


Figure 18: Estimated memory consumption as reported by the program analyzer.

7.5 Final Interpretation (Which Method Is Best?)

The test runs indicate that:

- **DFS:** Fast and low cost but provides the poorest options for decision quality due to the failure to model optimal counter-play; in fact, it will select moves that would lose if both players were playing perfectly.
- **Heuristic:** Fast and light weight; works well in many cases but retains its greedy characteristic (based on only one step scoring) and therefore cannot guarantee optimality across all states of play.
- **Minimax + Alpha-Beta:** Provides the highest quality of decisions made; ensures that the AI will not lose any game (plays to a win or draw) by performing an optimal adversarial search, however, has an extremely high computation cost during the earlier phases of the game.

Based on this testing, the heuristic search algorithm is the best algorithm to use when performances are critical; however, Minimax+AB remains the strongest optimal baseline comparison to the heuristic search.

Appendix A: Console Output Logs (Allegro CL)

Appendix A.1 — Full DFS (Mode 1) Log

```
=========================================
   TIC-TAC-TOE PERFORMANCE ANALYZER
=========================================
1. Uninformed (DFS)
2. Informed (Heuristic)
3. Informed (Minimax + AlphaBeta)
4. EXIT
-----------------------------------------
Select Option (1-4): 1

 1 | 2 | 3
-----------
 4 | 5 | 6
-----------
 7 | 8 | 9

Enter Move (1-9): 1

 X | 2 | 3
-----------
 4 | 5 | 6
-----------
 7 | 8 | 9

[AI STATS]
 > Algo:          DFS
 > Time:            0.0 s
 > Nodes:         9
 > Memory Est:    1. KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

```
 X | 2 | 3
-----------
 4 | O | 6
-----------
 7 | 8 | 9

Enter Move (1-9): 4

 X | 2 | 3
-----------
 X | O | 6
-----------
 7 | 8 | 9

[AI STATS]
 > Algo:          DFS
 > Time:            0.0 s
 > Nodes:         4
 > Memory Est:    .5 KB
 > Win Chance:    0% (Losing Path)
-----------------------------

 X | 2 | O
-----------
 X | O | 6
-----------
 7 | 8 | 9

Enter Move (1-9): 7
```

```
 X | 2 | O
-----------
 X | O | 6
-----------
 X | 8 | 9

==========================================
      FINAL PERFORMANCE REPORT
==========================================
 Total AI Moves:      2
 Perfect Decisions:   1
 AI FINAL ACCURACY:   50.%
==========================================

>>> RESULT: X WINS! <<<
```

Appendix A.2 — Full Heuristic (Mode 2) Log

```
=========================================
   TIC-TAC-TOE PERFORMANCE ANALYZER
=========================================
1. Uninformed (DFS)
2. Informed (Heuristic)
3. Informed (Minimax + AlphaBeta)
4. EXIT
-----------------------------------------
Select Option (1-4): 2

 1 | 2 | 3
-----------
 4 | 5 | 6
-----------
 7 | 8 | 9

Enter Move (1-9): 3

 1 | 2 | X
-----------
 4 | 5 | 6
-----------
 7 | 8 | 9

[AI STATS]
 > Algo:          HEURISTIC
 > Time:            0.0 s
 > Nodes:         8
 > Memory Est:    1. KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

```
 1 | 2 | X
-----------
 4 | O | 6
-----------
 7 | 8 | 9

Enter Move (1-9): 1

 X | 2 | X
-----------
 4 | O | 6
-----------
 7 | 8 | 9

[AI STATS]
 > Algo:          HEURISTIC
 > Time:          0.016 s
 > Nodes:         6
 > Memory Est:    .7 KB
 > Win Chance:    50% (Forced Draw)
-----------------------------

 X | O | X
-----------
 4 | O | 6
-----------
 7 | 8 | 9

Enter Move (1-9): 8
```

```
 X | O | X
-----------
 4 | O | 6
-----------
 7 | X | 9

[AI STATS]
 > Algo:          HEURISTIC
 > Time:          0.015 s
 > Nodes:         4
 > Memory Est:    .5 KB
 > Win Chance:    50% (Forced Draw)
-----------------------------

 X | O | X
-----------
 O | O | 6
-----------
 7 | X | 9

Enter Move (1-9): 6

 X | O | X
-----------
 O | O | X
-----------
 7 | X | 9
```

```
[AI STATS]
 > Algo:          HEURISTIC
 > Time:          0.016 s
 > Nodes:         2
 > Memory Est:    .2 KB
 > Win Chance:    50% (Forced Draw)
-----------------------------

 X | O | X
-----------
 O | O | X
-----------
 7 | X | O

Enter Move (1-9): 7

 X | O | X
-----------
 O | O | X
-----------
 X | X | O

=========================================
      FINAL PERFORMANCE REPORT
=========================================
 Total AI Moves:      4
 Perfect Decisions:   4
 AI FINAL ACCURACY:   100.%
=========================================

>>> RESULT: DRAW! <<<
```

Appendix A.3 — Full Minimax + AlphaBeta (Mode 3) Log

```
=========================================
   TIC-TAC-TOE PERFORMANCE ANALYZER
=========================================
1. Uninformed (DFS)
2. Informed (Heuristic)
3. Informed (Minimax + AlphaBeta)
4. EXIT
-----------------------------------------
Select Option (1-4): 3

 1 | 2 | 3
-----------
 4 | 5 | 6
-----------
 7 | 8 | 9

Enter Move (1-9): 3

 1 | 2 | X
-----------
 4 | 5 | 6
-----------
 7 | 8 | 9

[AI STATS]
 > Algo:          MINIMAX+AB
 > Time:          3.172 s
 > Nodes:         5119
 > Memory Est:    600. KB
 > Win Chance:    50% (Forced Draw)
-----------------------------
```

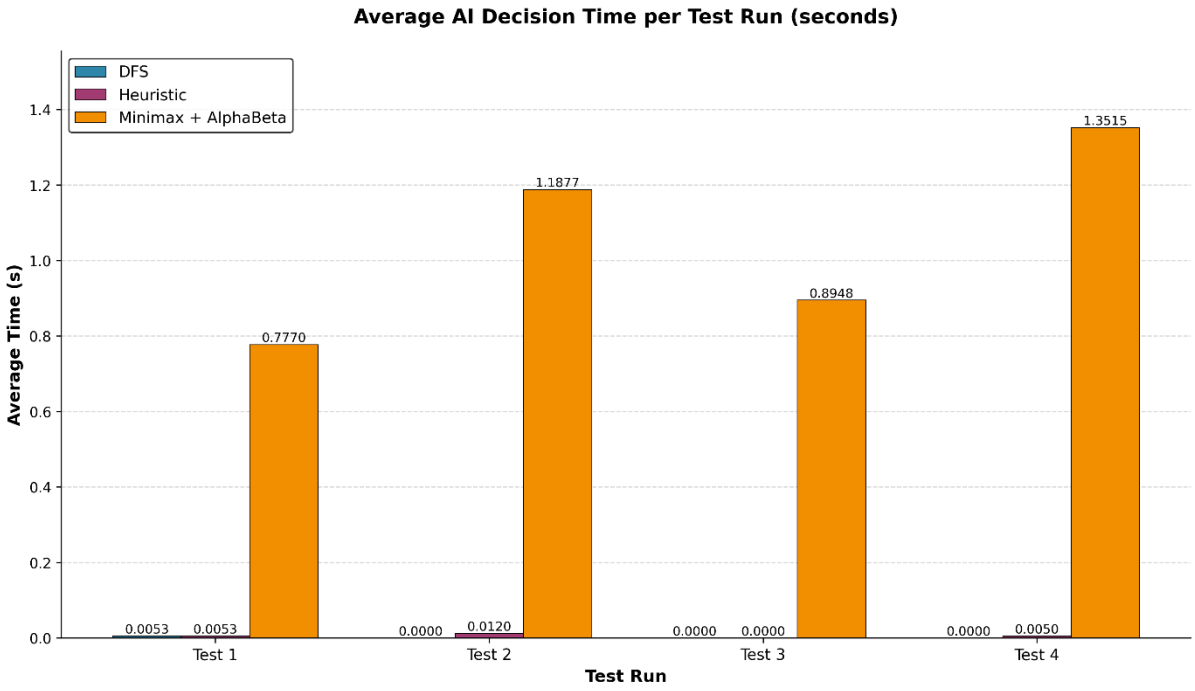
```
 1 | 2 | X
-----------
 4 | O | 6
-----------
 7 | 8 | 9

Enter Move (1-9): 7

 1 | 2 | X
-----------
 4 | O | 6
-----------
 X | 8 | 9

[AI STATS]
 > Algo:          MINIMAX+AB
 > Time:          0.281 s
 > Nodes:         433
 > Memory Est:    50. KB
 > Win Chance:    50% (Forced Draw)
-----------------------------

 1 | O | X
-----------
 4 | O | 6
-----------
 X | 8 | 9

Enter Move (1-9): 8
```

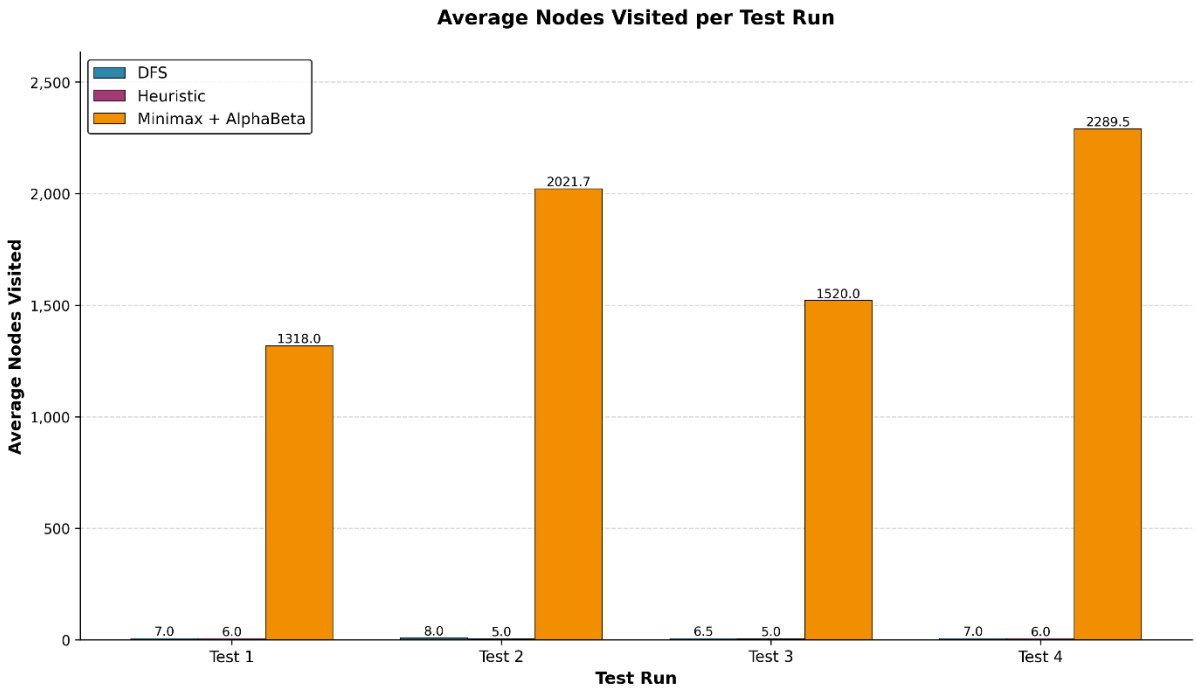
```
 1 | O | X
-----------
 4 | O | 6
-----------
 X | X | 9

[AI STATS]
 > Algo:          MINIMAX+AB
 > Time:          0.031 s
 > Nodes:         34
 > Memory Est:    4. KB
 > Win Chance:    50% (Forced Draw)
-----------------------------

 1 | O | X
-----------
 4 | O | 6
-----------
 X | X | O

Enter Move (1-9): 1

 X | O | X
-----------
 4 | O | 6
-----------
 X | X | O
```

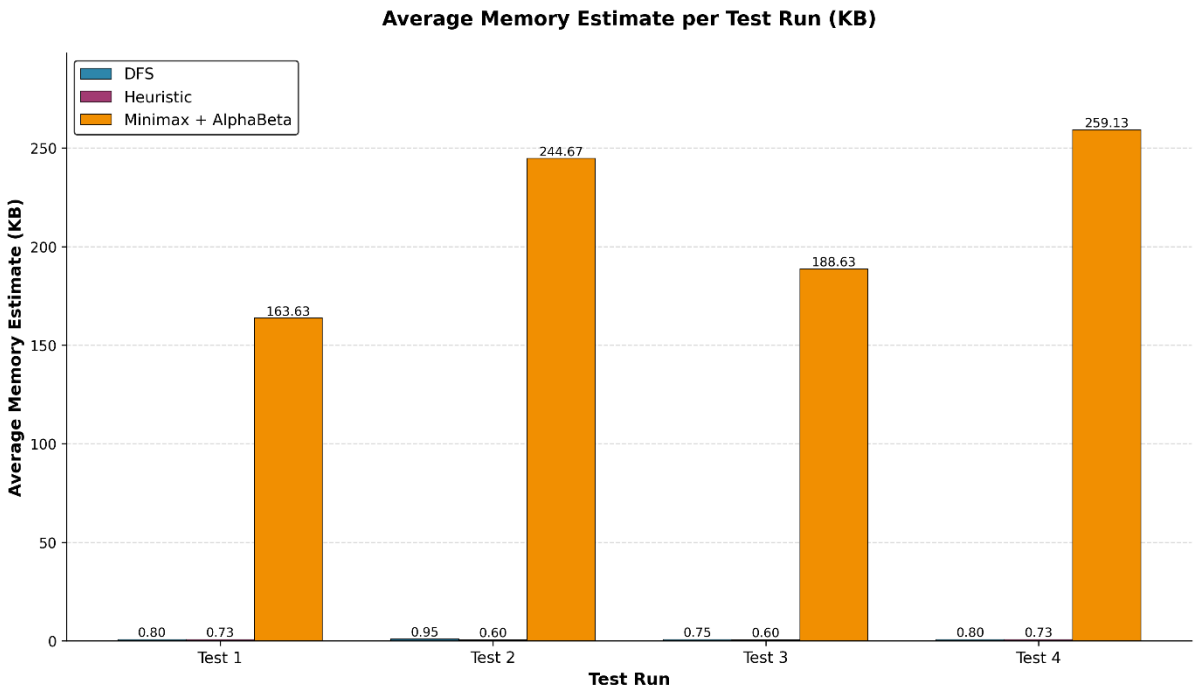
```
[AI STATS]
 > Algo:          MINIMAX+AB
 > Time:            0.0 s
 > Nodes:         4
 > Memory Est:    .5 KB
 > Win Chance:    50% (Forced Draw)
-----------------------------

 X | O | X
-----------
 O | O | 6
-----------
 X | X | O

Enter Move (1-9): 6

 X | O | X
-----------
 O | O | X
-----------
 X | X | O

=========================================
      FINAL PERFORMANCE REPORT
=========================================
 Total AI Moves:      4
 Perfect Decisions:   4
 AI FINAL ACCURACY:   100.%
=========================================

>>> RESULT: DRAW! <<<
```

Appendix B: 4 Test Runs per Algorithm

Avg Time / Max Time are computed from the AI move times printed in [AI STATS] for that test.
Avg Nodes / Max Nodes are from the Nodes values printed in [AI STATS].
Avg/Max Memory Est are from the Memory Est printed in [AI STATS].
Accuracy (%) is copied from the program's final report for that test.







Which one is better overall (based on these four runs each)?

Best for reliability: Minimax + Alpha-Beta.
Because in the 4 tests it had **100% accuracy every time**, **never lost**, and even got **an AI win** (plus 3 draws). It is the only method that consistently avoids losing across all runs.

Best for speed / low resource use: Heuristic.
It is far faster and lighter than Minimax, but it is **not always safe**.

Worst overall here: DFS.
It lost **every test**, with the lowest overall accuracy (**55.6%**), even though it is fast.

8. Conclusion

8.1 Summary of Project Achievements

Achieving the course goals, which include hands, on experience with decision trees and search algorithms, the present work has resulted in an intelligent agent that plays Tic, Tac, Toe using the Common Lisp language. We successfully connected the theoretical base of AI with its practical use by changing the real 3x3 game board into a LISP, readable format (linear lists) and state changes were done through symbolic processing.

Three different search methods were tried and tested, each of which was Uninformed Depth, First Search (DFS), an Informed Greedy Heuristic Search, and a Minimax algorithm with Alpha, Beta pruning. To test these methods, we made a performance analyzer that, among other parameters, checked the time of execution, the amount of memory used and the decision's optimality.

The main points can be summarized as follows:

Optimal: Minimax with Alpha, Beta pruning reached 100% accuracy and could thus be considered the perfect decision, making baseline, still, it was the most resource, demanding method with an average memory usage of 183.85 KB.

Efficiency: Greedy Heuristic search was the most efficient. Using a custom evaluation function that "blocked threats" and "controlled the center, " it is Best for speed / low resource use while being faster and using a negligible amount of memory, but it is **not always safe**.

Disadvantages of informed Search: DFS got only 50% correct answers during our trials which means that the game environment calls for either heuristics (domain, specific knowledge) or exhaustive search (Minimax).

8.2 Future Improvements and Recommendations

There are several domains of the solution that can be developed or improved in the next versions:

1. **Scalability to N x N Grids Installing:**

The existing solution is only for a 3x3 grid. One major improvement would be generalizing the state representation so it can manage larger boards (e.g., 4x4 or 5x5) or different winning conditions (e.g., Gomoku). However, with the rise of the branching factor, the standard Minimax algorithm would be beyond computational feasibility. To resolve this, we suggest using Monte Carlo Tree Search (MCTS), which is more efficient in larger search spaces than conventional alpha, beta pruning.

2. **Machine Learning for Heuristic Weights:**

   Thus far, the heuristic weights (for instance, ,1000 for opponent threats, +100 for AI threats) were mostly based on logical reasoning and trial, and error. Rather than depending on static, hard, coded hyperparameters, a future enhancement would be to create a Reinforcement Learning agent that "learns" these weight values by playing thousands of games against itself.

3. **User Interface (UI) Upgrade:**
   The present interaction mode was through the console output of Allegro CL. A graphical user interface (GUI) based on a Lisp web framework or a Python frontend coupled with Lisp logic, would be a terrific way for non-technical users to see the game visually and easily get involved.

4. **Iterative Deepening for DFS:**
   To boost the efficiency of the uninformed search, Iterative Deepening Depth, First Search (IDDFS) could be used. This way the algorithm can progressively explore deeper levels initially, thereby potential winning solutions can be found at shallower depths faster, and also it won't face the problem of standard DFS where it gets lost very deeply in the search tree without finding a goal.

9. References

Franz Inc. (2025). *Allegro Common Lisp documentation* [Computer software]. https://franz.com/support/documentation/

Graham, P. (1995). *ANSI Common Lisp*. Prentice Hall. http://www.paulgraham.com/acl.html

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson. http://aima.cs.berkeley.edu

Seibel, P. (2005). *Practical Common Lisp*. Apress. http://www.gigamonkeys.com/book/

Shannon, C. E. (1950). Programming a computer for playing chess. *Philosophical Magazine*, *41*(314), 256–275. https://doi.org/10.1080/14786445008521796